

50.037 Blockchain Technology

Decentralized Healthcare Application Final Report

Group XAE A-12:
Toh Hong Jing, Chua Xuan Athena,
Sng Kiat Loong Joel, Kevin Teng Jin Peng

January 8, 2025

Contents

1	Executive Summary	4
2	Introduction	4
2.1	Motivation	4
2.2	Problem Statement	4
2.3	Blockchain Use Case	4
2.3.1	Immutable Storage	4
2.3.2	Access Control	4
2.3.3	Decentralisation	5
2.3.4	Enhanced Interoperability	5
2.4	Application Overview	5
2.4.1	Stakeholders Analysis	5
2.4.2	Use Case Diagram	5
2.5	Design Considerations	6
2.5.1	Data Privacy and Security	6
2.5.2	Data Storage	6
3	System Design	7
3.1	Overview	7
3.2	Blockchain Layer	7
3.2.1	Solidity	7
3.2.2	Hardhat	7
3.3	Data Storage Layer	7
3.3.1	InterPlanetary File System(IPFS)	7
3.4	Frontend Layer	7
3.4.1	React	7
3.4.2	Metamask	7
3.5	System architecture	7
3.6	Data Flow Diagrams	8
4	Smart Contract Implementation	10
4.1	Contracts Overview	10
4.2	Design Principles	10
4.3	Class Diagram	10
4.4	PatientRegistry.sol	11
4.5	HealthcareProviderRegistry.sol	11

4.6	AccessControl.sol	12
4.7	MedicalRecords.sol	13
4.8	Gas Fees Analysis	13
4.8.1	Optimising Gas Usage	14
5	Security Analysis	16
5.1	IPFS Content Identifier(CID)	16
5.2	Encryption of Medical Records	16
5.3	Access Control	16
5.4	Performance	16
6	Test Cases	17
6.1	Test Case 1: Patient Registration	17
6.1.1	Patient Registration	17
6.1.2	Prevent Double registration	17
6.2	Test Case 2: Access Control	17
6.2.1	Healthcare provider access	17
6.2.2	Unauthorized party prevention	17
6.2.3	Revoking access to a previously approve doctor	17
6.2.4	Prevents data access without explicit approval	17
6.2.5	Allow a patient to approve access	17
6.2.6	Allows a provider to retrieve the encrypted key after access is approved	17
6.2.7	Prevents provider from retrieving encrypted key if access is not approved	17
6.2.8	Allows a provider to retrieve the CID after access is approved	17
6.2.9	Should handle multiple access requests and approvals correctly	18
6.2.10	Prevents unauthorized users from revoking access	18
6.2.11	Handles invalid data correctly	18
6.3	Test Case 3: Medical Record Upload	18
6.3.1	Allow authorized providers to upload medical records	18
6.3.2	Prevent unauthorized providers from uploading medical records	18
6.3.3	Allow patients to retrieve their own medical records	18
6.3.4	Prevent unauthorized access to retrieve a patient's medical records	18
6.3.5	Allow authorized providers to retrieve a patient's medical records	18
6.4	Test Case 4: Healthcare Provider Registration	18
6.4.1	Healthcare provider registration	18
6.4.2	Allows the admin to verify a registered provider	18
6.4.3	Prevents a provider from registering more than once	18
6.4.4	Prevents the admin from verifying a provider who is not registered	19
6.4.5	Prevents non-admin users from verifying a provider	19
6.4.6	Allows only the admin to access the list of registered providers	19
6.4.7	Ensure that only registered providers' public keys can be accessed	19
6.5	Test Case 5: Gas Usage	19
6.6	Test case data	20
7	Frontend Interface	21
7.1	Overview	21
7.2	Screenshots and Flow	21
7.2.1	Landing Page	21
7.2.2	Patient Registration	22
7.2.3	Patient Dashboard	23
7.2.4	Doctor Registration	27
7.2.5	Doctor Dashboard	28
7.2.6	Admin Dashboard	30
8	Reflection and Improvements	32

8.1	Limitations	32
8.2	Further Improvements	32
8.2.1	Data Encryption	32
8.2.2	Optimising Data Extraction	32
8.2.3	Gas Fees Optimisation	32
8.2.4	Data Amendment	33
9	Roles and responsibilities	33
10	Conclusion	33

1 Executive Summary

In this report, we present a decentralized application (DApp) that leverages blockchain technology to address the challenges of managing healthcare records internationally. This provides our users with a platform that is unbounded by geolocation, as well as the varying rules and regulations of different countries. Our system ensures secure, efficient, and patient-centric control over medical data with a low barrier to entry for ease of use. The following sections will detail the system design, smart contract implementation, frontend interface, and provide an analysis of gas usage and security considerations.

2 Introduction

2.1 Motivation

Healthcare systems worldwide struggle with challenges related to data silos, privacy, and control of patient information. Patient data has traditionally been stored in isolated databases across multiple providers and institutions, creating fragmented data silos. This lack of interoperability complicates data access and management, leading to scalability challenges. Over time, this fragmentation increases the risk of incomplete medical histories and medical errors, ultimately impacting the quality and efficiency of healthcare delivery systems. Furthermore, centralized storage introduces significant risks of data tampering and loss, which can compromise patient safety. Given the sensitive nature of personal and confidential health information, any breach or corruption of this data can have serious consequences for patient privacy, trust, and healthcare outcomes. Finally, patients often lack autonomy and control over their health data, as it is typically stored and managed by healthcare providers or third-party entities. This limited control prevents patients from easily accessing, sharing, or revoking access to their own medical information, hindering their ability to make informed decisions about their healthcare. Blockchain technology addresses these challenges by enabling a decentralized, tamper-proof ledger that enhances data security and privacy. This approach empowers patients with full ownership and control over their healthcare information, allowing them to manage access permissions and ensure transparency in how their data is used and shared.

2.2 Problem Statement

How we might utilize blockchain technology to create a decentralized application that allows safe and secure access to modify healthcare records of patients while providing greater autonomy to patients in controlling their own personal information and health records.

2.3 Blockchain Use Case

The specific use case of blockchain within the DApp revolves around the secure, transparent, and efficient management of medical records. By leveraging on blockchain technology, the DApp can offer immutable storage, robust access control, and the inherent benefits of decentralization in the healthcare domain. Below is a detailed breakdown of these blockchain features in the context of the DApp.

2.3.1 Immutable Storage

Blockchain ensures that once medical records are entered into the system, they cannot be altered or tampered with. Instead of storing actual data, medical records are stored as a unique cryptographic hash on the blockchain. Hence, any changes made to the data result in a change in the hash, making it apparent that the data has been tampered with. Additionally, each change or addition to the record creates a new block in the chain, making it easy to track every modification and preventing fraudulent alterations.

2.3.2 Access Control

Blockchain ensures that only authorized individuals can access specific records, it offers patients control over who views their records. Patients can grant and revoke a doctor's access to their data through smart

contracts. The blockchain also records every access request and the patient's consent or denial, ensuring an auditable trail.

2.3.3 Decentralisation

Medical records are stored across multiple blockchain nodes, ensuring that the data is not held in a single location. This means that no single entity or central server controls the data, reducing the risk of hacks, data breaches, or system outages. Furthermore, it allows for decentralised consent, allowing patients full autonomy over their data. The patient's decision to grant or revoke access is stored on the blockchain, and those decisions cannot be altered by third parties.

2.3.4 Enhanced Interoperability

Blockchain helps bridge the gap between different healthcare systems and providers. It ensures that medical records can be securely accessed by healthcare providers, regardless of the institution or geographic location.

2.4 Application Overview

This DApp aims to connect patients and healthcare providers from various institutions on a shared network, providing a range of features and functionalities to support the above use cases.

2.4.1 Stakeholders Analysis

Our application caters to two primary user groups: Patients and Healthcare Providers. Their primary actions of patients involve granting or denying access requests from healthcare providers. Patients prioritize having full control over their personal health data, ensuring privacy, and maintaining the security of their information. To meet these needs, the application provides a user-friendly interface that allows patients to easily manage access permissions. Healthcare Providers require timely access to patient data to support treatment and consultation. Their key actions are to submit access requests for specific patient records, as well as to view them. Providers seek an efficient and seamless user experience, as timely access to accurate, up-to-date medical records is essential for delivering effective patient care. Hence, the system is designed to facilitate smooth interactions and quick access to the necessary information.

2.4.2 Use Case Diagram

An overview of the interactions between the users, as well as each component of the application is illustrated below.

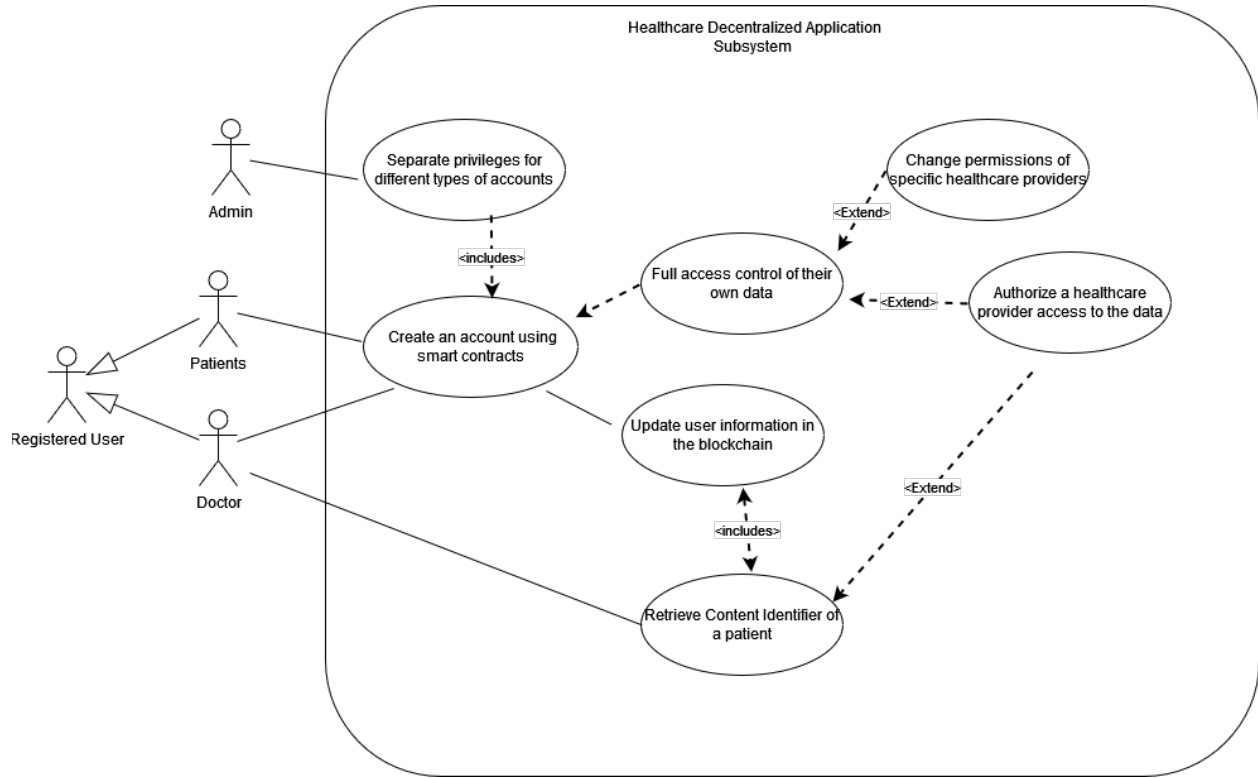


Figure 1: Use Case Diagram

2.5 Design Considerations

2.5.1 Data Privacy and Security

Since medical records are highly sensitive, all data must be encrypted. We will be using strong encryption algorithms such as AES256 to encrypt the records before storing them. This ensures that only authorized individuals can access the data.

We also need to implement granular access control mechanisms to restrict who can view certain medical data. This includes using smart contracts to enforce access permissions based on user roles as well as specific file records.

2.5.2 Data Storage

Storing large medical records directly on the blockchain could be inefficient and costly. Hence, we decided to go with off-chain storage solutions such as InterPlanetary File System(IPFS). This allows us to store large files such as PDFs separately from the chain. Each file on IPFS generates a unique hash that represents the record without revealing its content. These hashes are stored on the blockchain as references of the actual medical records.

3 System Design

3.1 Overview

For our DApp, the back-end will be made using well-established crypto libraries from online sources, relying on the Ethereum network to run our private blockchain, while interfacing with the frontend which will be coded in the React framework and running on an instance of a mock Ethereum network in Metamask to simulate blockchain interaction.

3.2 Blockchain Layer

3.2.1 Solidity

For the application framework, Solidity will be used to create the smart contracts used by both the patients and doctors as the library includes high-level functions including smart contract creation and interaction with each other.

3.2.2 Hardhat

In our deployment of smart contracts, we will be using the Hardhat development tool to test our functions in the contracts, including unit tests of each modifier and operation, which will be managed by the Chai framework.

3.3 Data Storage Layer

3.3.1 InterPlanetary File System(IPFS)

Data states, such as registered accounts and respective medical records, will be saved in a distributed and decentralized file system called the InterPlanetary File System (IPFS), where data are transferred to a peer-to-peer network.

3.4 Frontend Layer

3.4.1 React

The website will be using React frameworks with each page coded in Javascript where patients are able to interact with our application. Pages of the DApp will include a landing page where users are able to sign up and log in with their wallets, a registration and profile page for both patients and doctors and finally an admin dashboard where they have elevated privileges to help debug user issues.

3.4.2 Metamask

Our application will be able to interface with hot wallets registered in Metamask, enabling users to connect directly to the Ethereum blockchain. For testing and debugging purposes we will also create a few test accounts to test gas fees and transactions in a SepoliaETH mock network

3.5 System architecture

The architecture diagram is illustrated below for a clearer view of the system.

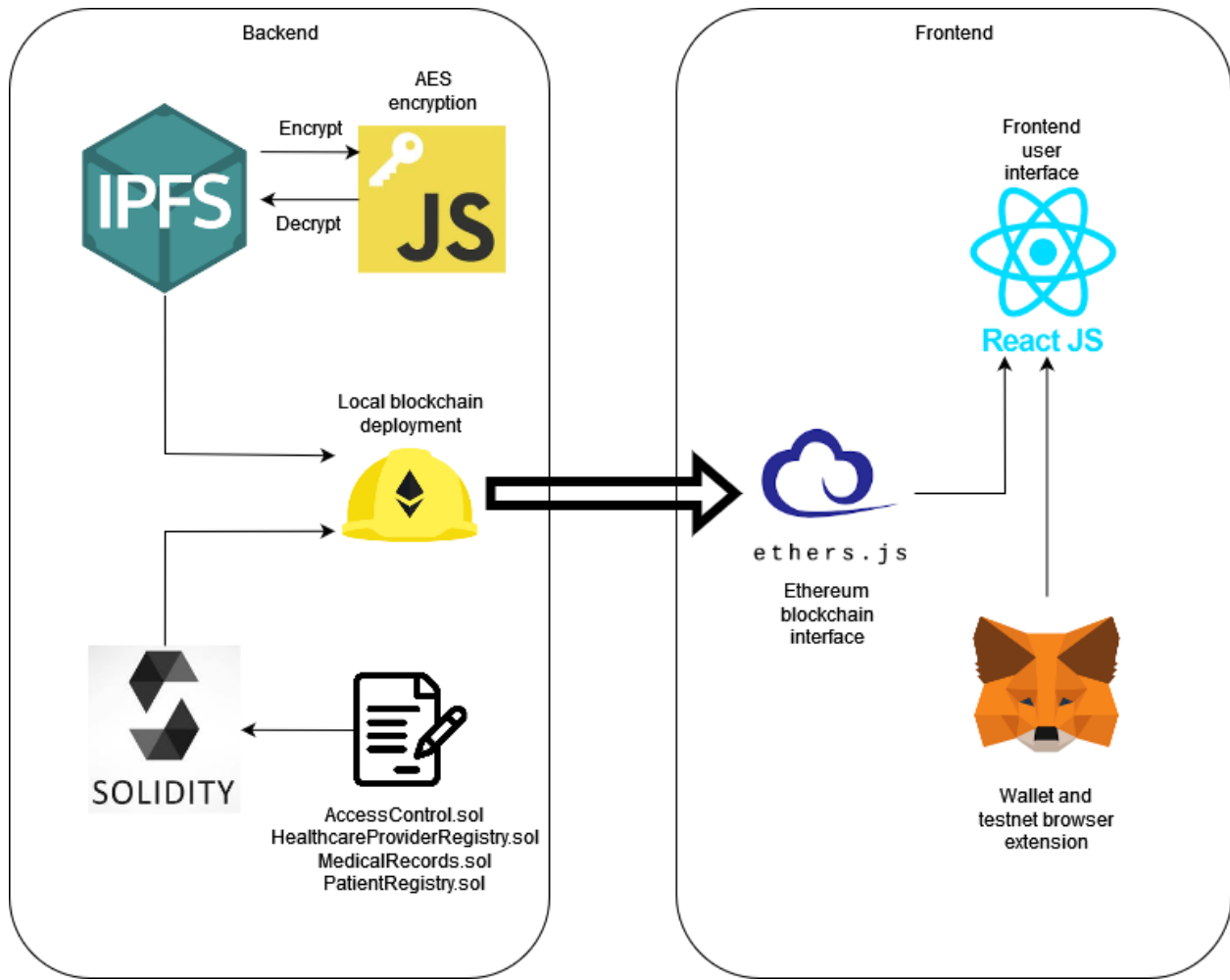


Figure 2: System Architecture

3.6 Data Flow Diagrams

The following diagram illustrates the data flow between the users as well as various components of the DApp. Firstly, patients and doctors create their accounts on the application which calls its respective smart contracts. Patients are able to upload their medical records in the form of PDF files as well. These files will be encrypted and stored on the IPFS. The IPFS generates a hash of the file, known as its content identifier (CID), which will be stored on the blockchain. Doctors can submit requests to access a patient's medical records, with each request recorded on the blockchain for transparency and traceability. Patients are notified of these requests via the application, where they can approve or reject them. Their actions, whether approval or rejection, are also logged on the blockchain, ensuring a comprehensive audit trail for data access and privacy management.

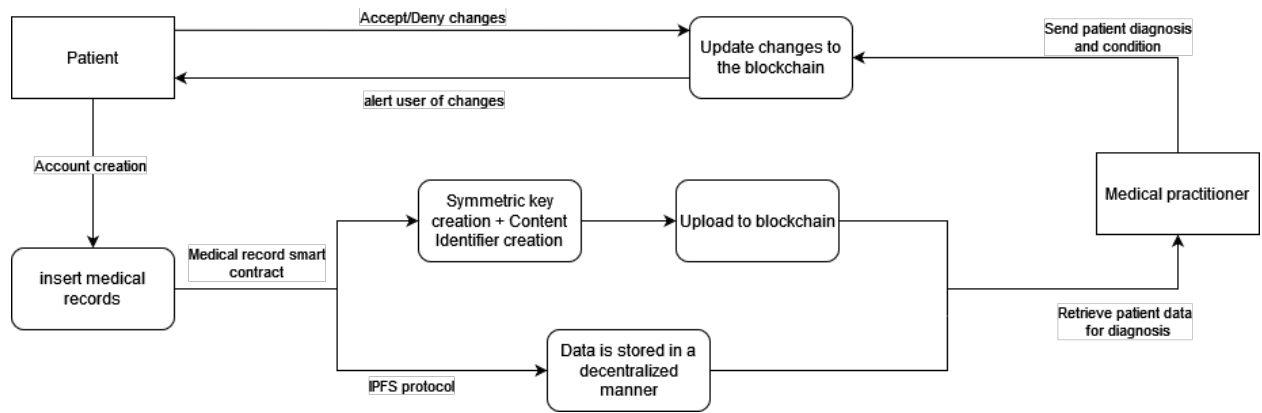


Figure 3: Data Flow Diagram

4 Smart Contract Implementation

4.1 Contracts Overview

The development of this DApp revolves around creating a secure platform that empowers patients and their healthcare providers to manage medical records with privacy and transparency. The smart contracts form the core components of this DApp, and they help facilitate some of the most important functions of our app such as:

- Patient registration
- Healthcare provider registration
- Access control
- Medical Record management

This section outlines the thought process behind the creation of the smart contracts, explains design decisions, and provides detailed explanations of the code implemented.

4.2 Design Principles

The primary objectives guiding the smart contract development are:

- Protect sensitive medical data from unauthorised access
- Give patients full control over their medical records in terms of giving permissions
- Eliminate reliance on centralised authorities by using Blockchain Technology
- Implement contracts with modularity in mind, with clear separation of concerns for maintainability and scalability
- Ensures an immutable audit trail of all transactions for accountability
- Ensure that attacks that exploit loopholes in smart contracts are impossible
- Gas limit for each contract activation is reasonable

4.3 Class Diagram

The diagram below illustrates an overview of the smart contracts, its attributes, modifiers and functions.

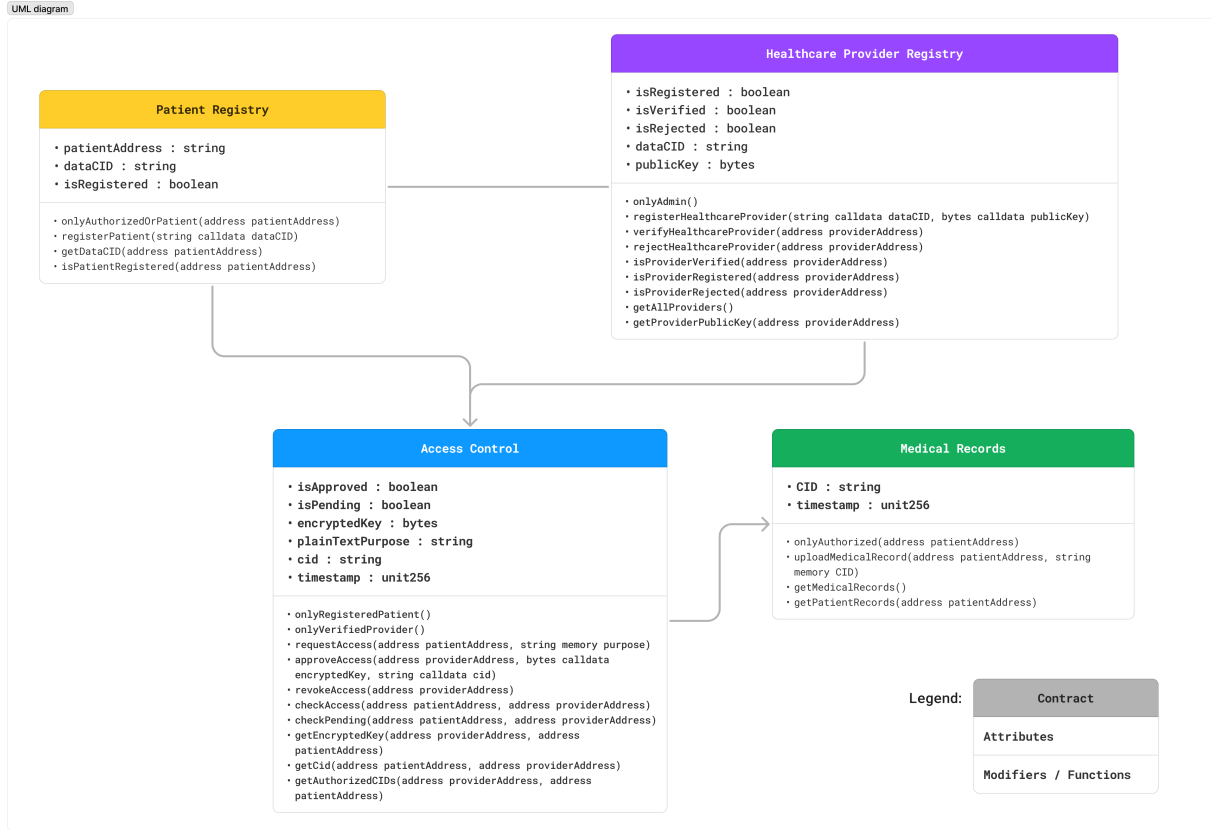


Figure 4: Smart Contract Class Diagram

4.4 PatientRegistry.sol

The PatientRegistry.sol smart contract manages patient registration.

Key Features:

- **registerPatient:** Allows patients to register by providing their address and a Content Identifier (CID) pointing to their encrypted off-chain data.
- **isAuthorized:** Verifies if a healthcare provider is authorized to access a patient's data.
- **getDataCID:** Retrieves the off-chain data CID of a patient, accessible only by the patient or their authorized providers.
- **isPatientRegistered:** Checks if a patient is registered in the system.

Security:

- **onlyAuthorizedOrPatient** modifier: Ensures that only the patient or an authorized provider can access sensitive functions.

Events:

- **PatientRegistered:** Emitted when a patient registers in the system.

4.5 HealthcareProviderRegistry.sol

The HealthcareProviderRegistry.sol smart contract manages the registration and verification of healthcare providers. This enables providers to register themselves and allows the administrator to verify registered providers.

Key Features:

- **registerHealthcareProvider:** Allows healthcare providers to self-register by uploading their personal data onto IPFS and generating a unique CID
- **verifyHealthcareProvider:** Enables the administrator to verify registered providers, marking them as trusted
- **rejectHealthcareProvider:** Enables the administrator to reject a healthcare provider
- **isProviderRegistered:** Checks whether a provider is registered
- **isProviderVerified:** Checks whether a provider is verified

Security:

- **onlyAdmin:** A modifier that restricts specific actions, such as verification and rejection, to the admin

Events:

- **ProviderRegistered:** Emitted when a provider successfully registers
- **ProviderVerified:** Emitted when a provider is verified by the admin
- **ProviderDataUpdated:** Emitted when a registered provider updates their off-chain data CID
- **ProviderRejected:** Emitted when a provider is rejected by the admin

4.6 AccessControl.sol

The `AccessControl.sol` smart contract manages the permissions of healthcare providers to access patient data in a decentralized way. It integrates with external contracts, `PatientRegistry` and `HealthcareProviderRegistry`, to ensure that only registered patients and verified providers interact with the system. The contract uses a mapping to track access requests, represented by a struct that stores the approval status and a hash of the purpose of access.

Key Features:

- **requestAccess:** Verified providers can request access to a patient's data for a specific purpose
- **approveAccess:** Registered patients can approve a provider's access request
- **revokeAccess:** Patients can revoke previously granted access
- **checkAccess:** Allows anyone to verify if a provider has access to a patient's data

Security:

- **onlyRegisteredPatient** restrict actions to registered patients, ensuring that patients maintain control over their medical data
- **onlyVerifiedProvider** restrict actions to verified providers only

Events:

- **AccessRequested:** Emitted when a healthcare provider requests access to a patient's data
- **AccessApproved:** Emitted when a patient approves a healthcare provider's access request
- **AccessRevoked:** Emitted when a patient revokes a healthcare provider's access to their data

4.7 MedicalRecords.sol

The `MedicalRecords.sol` smart contract provides a secure and transparent system for storing and accessing patients' medical records. It integrates with the `AccessControl` contract to enforce strict access permissions, ensuring that only authorized users (patients or approved healthcare providers) can upload or retrieve records.

Key Features:

- `uploadMedicalRecord`: Allows authorized users (patients or their approved providers) to upload a medical record by providing a hashed or encrypted Content Identifier (CID)
- `getMedicalRecords`: Enables patients to retrieve all their own medical records
- `getPatientRecords`: Allows authorized users (patients or approved providers) to access a patient's records

Security:

- `onlyAuthorized`: A modifier that ensures only the patient or a provider with explicit access approval can interact with restricted functions.
- Data Privacy: Only hashed or encrypted CIDs of medical records are stored on-chain, with the actual records stored off-chain for enhanced security.

Events:

- `MedicalRecordUploaded`: Emitted when a medical record is successfully uploaded, providing transparency and traceability.

4.8 Gas Fees Analysis

There are a few functions in the smart contracts that interact with the blockchain and will require users to pay gas fees. These functions are:

- `requestAccess`
- `approveAccess`
- `revokeAccess`
- `registerPatient`
- `registerHealthcareProvider`
- `verifyHealthcareProvider`
- `rejectHealthcareProvider`
- `uploadMedicalRecord`
- Deploying the `PatientRegistry` contract
- Deploying the `HealthcareProviderRegistry` contract
- Deploying the `AccessControl` contract
- Deploying the `MedicalRecord` contract

The formula for calculating the gas fee in Ethereum is given by this equation:

$$\text{Gas Fee (ETH)} = \text{Gas Used} \times \text{Gas Price (in ETH)} \quad (1)$$

Assuming the price of ETH at USD 3730.

Function	Gas Used	20 gwei (Normal)	50 gwei (Moderate)	100 gwei (High)	200 gwei (Peak)
requestAccess	64123	\$4.78	\$11.96	\$23.91	\$47.82
approveAccess	101435	\$7.57	\$18.92	\$37.83	\$75.65
revokeAccess	35379	\$2.64	\$6.61	\$13.23	\$26.45
registerPatient	91248	\$6.81	\$17.03	\$34.06	\$68.11
registerHealthcareProvider	138353	\$10.32	\$25.79	\$51.57	\$103.15
verifyHealthcareProvider	30794	\$2.30	\$5.74	\$11.48	\$22.96
rejectHealthcareProvider	30592	\$2.28	\$5.70	\$11.40	\$22.81
uploadMedicalRecord	100460	\$7.49	\$18.72	\$37.43	\$74.86
deploying PatientRegistry	567703	\$42.37	\$105.92	\$211.84	\$423.68
deploying HealthcareProviderRegistry	718505	\$53.66	\$134.16	\$268.32	\$536.64
deploying AccessControl	942616	\$70.40	\$176.01	\$352.01	\$704.02
deploying MedicalRecords	524356	\$39.16	\$97.90	\$195.79	\$391.58

Table 1: Gas Costs for Various Functions and Deployments at Different Gwei Levels (USD)

4.8.1 Optimising Gas Usage

Gas fees play a critical role in the adoption and scalability of blockchain-based platforms. It is important to ensure that these fees remain affordable, as excessively high gas fees can negatively impact user adoption and operational scalability. For example, during periods of peak network demand, high gas fees can deter new users from engaging with the application. A notable instance is the deployment of the **AccessControl** contract, which incurs a cost of \$704 during peak network conditions. Such a high cost may be prohibitive for smaller organizations or individual users with limited financial resources.

In addition to deployment costs, frequent function calls can also result in substantial cumulative expenses. For instance, the **uploadMedicalRecord** function incurs a gas fee of \$74.86 under high gas price conditions (100 gwei). For users who need to execute this function regularly, these recurring costs may make the platform financially unsustainable and seem less attractive to them, particularly to users requiring frequent interactions.

Hence, to address these challenges, it is essential to optimise gas usage through improved contract design and implementation. An example of this would be to set gas limits. Setting gas limits not only protects users from excessive fees but also minimizes the risk of out-of-gas errors that could occur due to unforeseen circumstances, such as network congestion or increased computational demands. Furthermore, carefully chosen limits provide a buffer to accommodate fluctuations in gas prices, particularly during peak traffic periods when fees can spike significantly.

The table below shows our recommended gas limits for each function and contract deployment to provide some headroom to account for the increased gas prices during peak traffic periods.

Function	Gas Used	Recommended Gas Limit
requestAccess	64123	70000
approveAccess	101435	120000
revokeAccess	35379	40000
registerPatient	91248	100000
registerHealthcareProvider	138353	150000
verifyHealthcareProvider	30794	35000
rejectHealthcareProvider	30592	35000
uploadMedicalRecord	100460	150000
Deploying PatientRegistry	567703	600000
Deploying HealthcareProviderRegistry	718505	750000
Deploying AccessControl	942616	1000000
Deploying MedicalRecords	524356	550000

Table 2: Gas Usage and Recommended Limits for Smart Contract Functions and Deployments

5 Security Analysis

5.1 IPFS Content Identifier(CID)

To ensure the confidentiality of patient data, we have used the InterPlanetary File System (IPFS) for secure storage of medical records. When a patient's medical record is uploaded to IPFS, it generates a unique Content Identifier (CID). This CID serves as a cryptographic fingerprint of the data, and is subsequently stored on the blockchain, creating a tamper-evident and transparent record. Since the CID is directly tied to the data itself, any modifications to the underlying record will result in a completely different CID.

5.2 Encryption of Medical Records

The uploaded medical records are encrypted with Advanced Encryption Standard (AES) before being uploaded to IPFS. This ensures that even if the records are intercepted on IPFS, the content remains unreadable without the decryption key. Symmetric keys are derived using the patient's master password and a static salt. This ensures that the encryption key is never stored explicitly anywhere, but can be securely regenerated when needed. When granting access, a secure ephemeral key exchange is implemented using the elliptic curve secp256k1. This generates a shared secret between the patient and the doctor, which is then used to derive an AES encryption key through padding. The derived AES key is unique to this session, and is used to encrypt the symmetric key (which was used to encrypt the medical records). The encrypted symmetric key is then sent along with the patient's ephemeral public key to the healthcare provider. By doing so, we ensure that only the intended recipient can decrypt the symmetric key.

5.3 Access Control

Patients have complete control over who can access their medical records, ensuring that sensitive information is only shared with trusted parties. Through our web interface, patients can review and manage access requests from healthcare providers, with details such as the requester's identity and purpose of access clearly displayed. They can approve or deny requests and selectively share specific records identified by its respective CIDs. Furthermore, patients can revoke access at any time, providing ongoing control over their data. All access requests, approvals, and revocations are securely recorded on the blockchain, creating an immutable and tamper-proof log of interactions. This blockchain-based system ensures transparency and accountability, as every action is traceable and auditable. By decentralizing trust, the blockchain removes reliance on a single entity to maintain accurate logs, reducing the risk of tampering. This approach empowers patients to safeguard their privacy and enhances trust in that data-sharing processes.

5.4 Performance

Content identifiers contain the hash and codec of the data, which is presented in a string or binary format, thus making it relatively small and easy to compute. This makes our application lightweight and scalable.

6 Test Cases

6.1 Test Case 1: Patient Registration

6.1.1 Patient Registration

This tests if a patient is registered after the registration process by checking if their CID is generated and linked to their account upon creation.

6.1.2 Prevent Double registration

This tests if a registered patient is able to register the same user information again by checking if a CID is already attached to the account.

6.2 Test Case 2: Access Control

6.2.1 Healthcare provider access

This tests if a healthcare provider is able to access patient data if they have the correct public key and insert it into the correct registry. The test will simulate a provider asking for access for consultation purposes, which the system will check the access control if the provider is allowed to view the contents, which it would notice that the provider has yet to be approved thus returning a false boolean.

6.2.2 Unauthorised party prevention

This tests if a medical provider is not verified when they request access to a patients data, they are unable to access that patients contents.

6.2.3 Revoking access to a previously approve doctor

This tests if a registered provider who previously had access to a patient's medical record would have their privileges revoked after the patient changes the access permissions of said provider by calling the revokeAccess function.

6.2.4 Prevents data access without explicit approval

This tests if upon registering and being verified medical provider would have no access to any medical records prior to approval by their patients even though they have access to the patients CID public key.

6.2.5 Allow a patient to approve access

This tests when a patient has given the correct access control to their medical provider and the provider is also registered in the system, the provider should have the correct privileges to access patient medical records by checking if hasAccess is true.

6.2.6 Allows a provider to retrieve the encrypted key after access is approved

This tests if a registered and verified provider is able to retrieve the private key of their patients after approval is granted.

6.2.7 Prevents provider from retrieving encrypted key if access is not approved

This tests that the provider is not able to retrieve the private key if they are not given approval by their patients even though they know what the patients public key is.

6.2.8 Allows a provider to retrieve the CID after access is approved

This tests if CID is retrievable by healthcare providers after verification and approval is given by the patients.

6.2.9 Should handle multiple access requests and approvals correctly

This tests when patients have given multiple access to different medical providers, the providers should receive the requests individually with the correct CID attached as both have unique CIDs for each provider.

6.2.10 Prevents unauthorized users from revoking access

This tests if the correct account type has the necessary privileges to revoke access to a patients data.

6.2.11 Handles invalid data correctly

This tests if the access control function is able to handle an invalid CID input.

6.3 Test Case 3: Medical Record Upload

6.3.1 Allow authorized providers to upload medical records

This tests if only authorized providers are allowed to upload medical records given the correct CID, access control and private key

6.3.2 Prevent unauthorized providers from uploading medical records

This test if providers are yet to be authorized, they are unable to upload medical records to the blockchain.

6.3.3 Allow patients to retrieve their own medical records

This tests if patients are able to retrieve their own data from the blockchain given the correct inputs such as correct CID, access control and verified registration

6.3.4 Prevent unauthorized access to retrieve a patient's medical records

This test if the access control in patient data is working correctly by rejecting those without sufficient privileges to be denied access such as not being verified or not given the correct access control by their patients.

6.3.5 Allow authorized providers to retrieve a patient's medical records

This tests the function for retrieval of a patient medical records by providing the correct fields for the function to correctly work. The test case includes an authorized private and public key which is recognised in the system. Following this, the `getPatientRecord` operation should successfully return a field.

6.4 Test Case 4: Healthcare Provider Registration

6.4.1 Healthcare provider registration

This tests the healthcare registration contract functionality if a healthcare provider is able to register themselves in the system and the correct public keys are generated and stored correctly.

6.4.2 Allows the admin to verify a registered provider

This tests if admins accounts are able to verify the account is under healthcare provider using the providers public key and checking if the stored public key is similar in the system.

6.4.3 Prevents a provider from registering more than once

This tests if a provider has generated a public key previously, the system is able to check if a similar key has already been stored in the system beforehand and should deny their request for a second registration.

6.4.4 Prevents the admin from verifying a provider who is not registered

This tests the admin account should be unable to verify an unregistered provider.

6.4.5 Prevents non-admin users from verifying a provider

This tests accounts that are classified as provider or patient are unable to call verifyHealthcareProvider function.

6.4.6 Allows only the admin to access the list of registered providers

This tests only admin accounts are able to call getAllProviders function.

6.4.7 Ensure that only registered providers' public keys can be accessed

This tests medical providers who are not yet verified by the admin is unable to get the public keys.

6.5 Test Case 5: Gas Usage

```
Gas Usage Tests
Gas used for requestAccess: 64123
✓ should calculate gas usage for requestAccess
Gas used for approveAccess: 101435
✓ should calculate gas usage for approveAccess
Gas used for revokeAccess: 35379
✓ should calculate gas usage for revokeAccess
Gas used for registerPatient: 91248
✓ should calculate gas usage for registerPatient
Gas used for registerHealthcareProvider: 138353
✓ should calculate gas usage for registerHealthcareProvider
Gas used for verifyHealthcareProvider: 30794
✓ should calculate gas usage for verifyHealthcareProvider
Gas used for rejectHealthcareProvider: 30592
✓ should calculate gas usage for rejectHealthcareProvider
Gas used for uploadMedicalRecord: 100460
✓ should calculate gas usage for uploadMedicalRecord
Gas used for deploying PatientRegistry: 567703
✓ should calculate gas usage for deploying PatientRegistry
Gas used for deploying HealthcareProviderRegistry: 718505
✓ should calculate gas usage for deploying HealthcareProviderRegistry
Gas used for deploying AccessControl: 942616
✓ should calculate gas usage for deploying AccessControl
Gas used for deploying MedicalRecords: 524356
✓ should calculate gas usage for deploying MedicalRecords
```

Figure 5: Gas Usage

6.6 Test case data

```
● hongjingtoh@Hong's-MacBook-Air-2 Healthcare-DApp-hardhat % npx hardhat test

AccessControl
  ✓ should allow a provider to request access
  ✓ should prevent unverified provider from requesting access
  ✓ should allow a patient to revoke access
  ✓ should prevent provider from accessing data without approval
  ✓ should allow a patient to approve access
  ✓ should allow a provider to retrieve the encrypted key after access is approved
  ✓ should prevent provider from retrieving encrypted key if access is not approved
  ✓ should allow a provider to retrieve the CID after access is approved
  ✓ should handle multiple access requests and approvals correctly
  ✓ should prevent unauthorized users from revoking access
  ✓ should correctly identify authorized providers
  ✓ should handle invalid data correctly

HealthcareProviderRegistry
  ✓ should allow a provider to register
  ✓ should allow the admin to verify a provider
  ✓ should not allow a provider to register twice
  ✓ should prevent verifying an unregistered provider
  ✓ should prevent non-admin from verifying a provider
  ✓ should not allow a rejected provider to be verified
  ✓ should not allow a rejected provider to register again
  ✓ should only allow admin to retrieve all providers
  ✓ should not allow public key retrieval for unregistered providers

MedicalRecords
  ✓ should allow authorized providers to upload medical records
  ✓ should prevent unauthorized access to upload medical records
  ✓ should allow patients to retrieve their own medical records
  ✓ should prevent unauthorized access to retrieve a patient's records
  ✓ should allow authorized provider to retrieve patient's medical records

PatientRegistry
  ✓ should allow a patient to register
  ✓ should not allow a patient to register twice

28 passing (691ms)
```

Figure 6: Test case data

7 Frontend Interface

7.1 Overview

The application is split into three use cases: Doctor, Patient and Admin. The following will display the flow for each role and the actions available on each page of the application.

7.2 Screenshots and Flow

7.2.1 Landing Page

Users select their role as Doctor, Patient or Admin and proceed to the login or registration page. Users have to connect their wallet before proceeding.

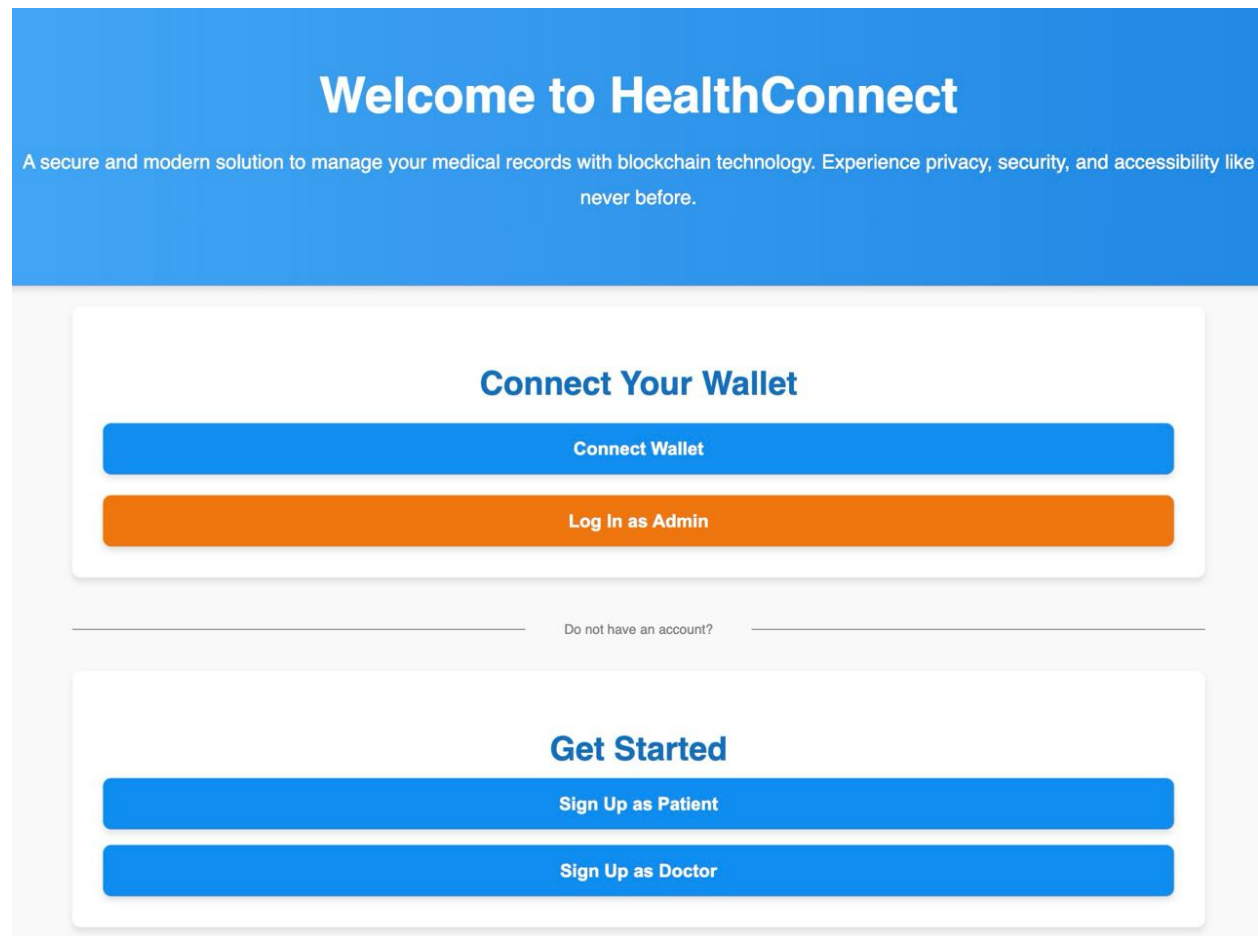


Figure 7: Landing Page

7.2.2 Patient Registration

Patients fill in their personal information for registration.

Patient Registration

Submit your personal information to register. This data will be securely stored off-chain.

Name:

Date of Birth:



Address:

MedicalHistory:

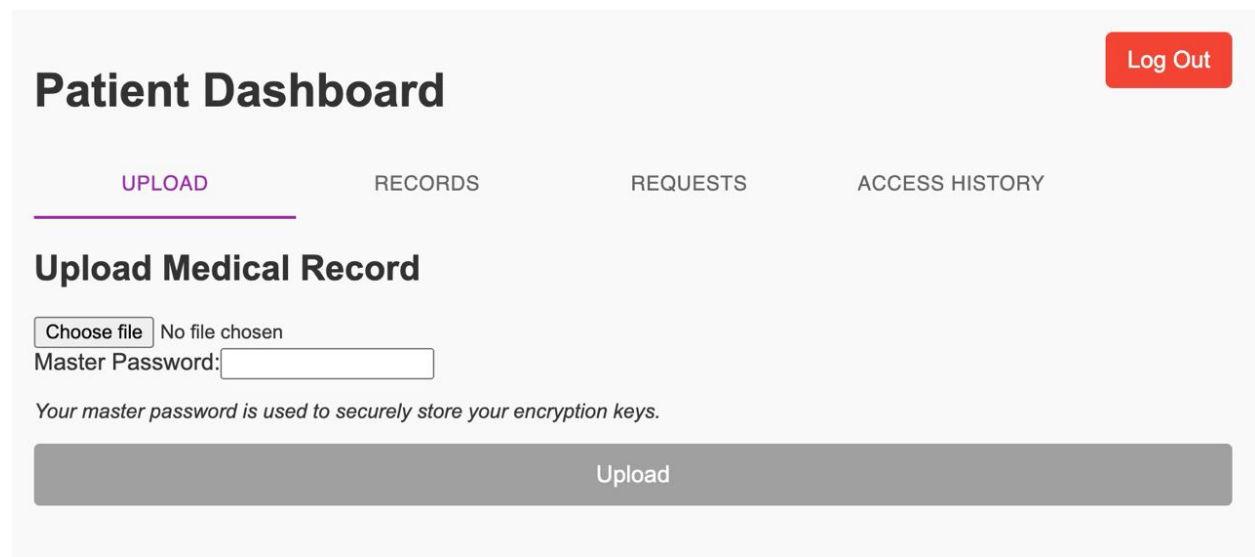
Contact:

Register

Figure 8: Patient Registration

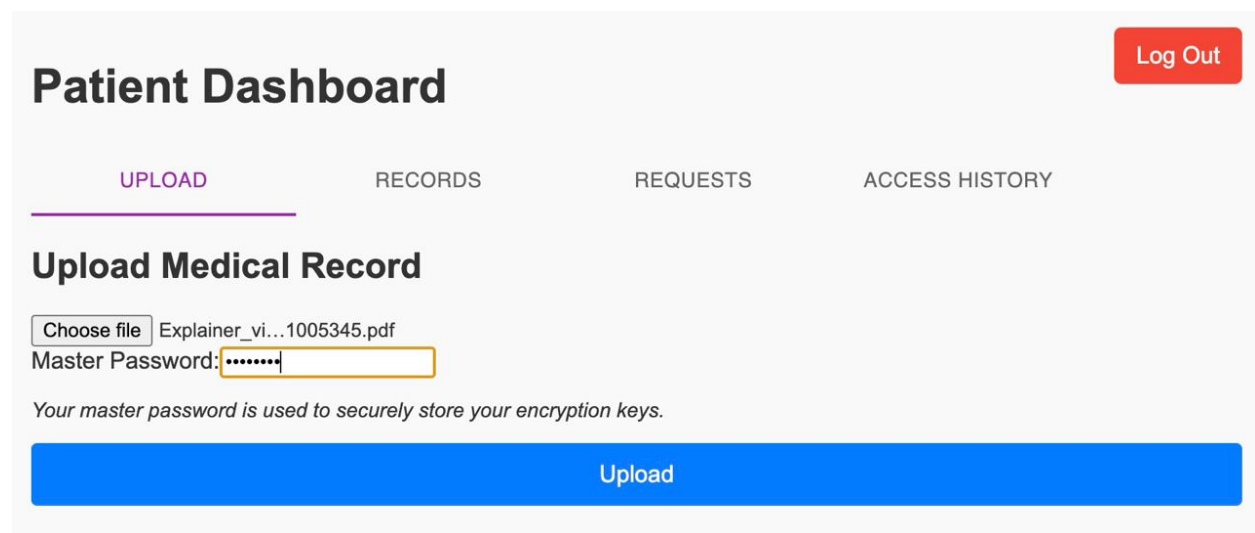
7.2.3 Patient Dashboard

The patient dashboard has four separate tabs for the following purposes: managing access permissions, viewing access logs, uploading medical records and viewing the list of medical records uploaded. Before uploading a file, the patient is required to enter their master password, which will be used to encrypt the file before storing it on the IPFS. Under Medical Records, the CID of the specific record together with the corresponding timestamp will be displayed to the user. Under Requests, information regarding the request will be displayed along with the 'Approve' or 'Reject' button. The final state of the request will be displayed with the option to revoke the request. Under Access history, patients can view a comprehensive list of healthcare providers who have been granted or revoked access to their medical records. This list includes timestamps for each action, providing patients with a clear and transparent record of when access permissions were granted or withdrawn..



This screenshot shows the 'Patient Dashboard' interface. At the top right is a red 'Log Out' button. Below the title are four tabs: 'UPLOAD' (highlighted with a purple underline), 'RECORDS', 'REQUESTS', and 'ACCESS HISTORY'. The main heading is 'Upload Medical Record'. Below it is a file selection area with a 'Choose file' button and the text 'No file chosen'. A 'Master Password:' label is followed by an empty text input field. A note states: 'Your master password is used to securely store your encryption keys.' At the bottom is a wide, grey 'Upload' button.

Figure 9: Patient Dashboard



This screenshot shows the 'Patient Dashboard' interface after a file has been selected. The 'Log Out' button remains in the top right. The 'UPLOAD' tab is still highlighted. The file selection area now shows 'Choose file' and 'Explainer_vi...1005345.pdf'. The 'Master Password:' label is followed by a text input field containing seven dots. The same note about the master password is present. The 'Upload' button at the bottom is now blue.

Figure 10: Patient Dashboard

Patient Dashboard

[Log Out](#)[UPLOAD](#)[RECORDS](#)[REQUESTS](#)[ACCESS HISTORY](#)

Your Medical Records

- **CID:** QmVcGHrqbKRYZjZr3eeQyYqvnPQKC9MxdVXGbgFhhF5ET

Timestamp: 12/12/2024, 19:20:24

- **CID:** QmUknbPALncLEffbXueBvCnzAArF8LdGrqGzJmEwvkT9iq

Timestamp: 12/12/2024, 19:20:39

Figure 11: Patient Dashboard (Records)

Patient Dashboard

[Log Out](#)[UPLOAD](#)[RECORDS](#)[REQUESTS](#)[ACCESS HISTORY](#)

Manage Permissions

Name	Type	Access	Action
------	------	--------	--------

Grant Access

Master Password:

Your master password is used to decrypt your encryption keys.

Doctor Name	Date Requested	Purpose	Current Access	Select Medical Records (CIDs)	Actions
Doctor Tan	12/12/2024, 07:22 pm	get medical record A	No Authorised Records	QmVcGHrqbKRYZjZr3eeQyYqvnPQKC9MxdVXGbgFhhF5ET QmUknbPALncLEffbXueBvCnzAArF8LdGrqGzJmEwvkT9iq	Approve Reject

Figure 12: Patient Dashboard (Requests)

Patient Dashboard

Log Out

UPLOAD

RECORDS

REQUESTS

ACCESS HISTORY

Manage Permissions

Name	Type	Access	Action		
Grant Access					
Master Password:					
Your master password is used to decrypt your encryption keys.					
Doctor Name	Date Requested	Purpose	Current Access	Select Medical Records (CIDs)	Actions
Doctor Tan	12/12/2024, 07:22 pm	get medical record A	No Authorised Records	QmVcSHmx8KrYz4Z/3esQyYqvnP0Kc3MxdVXGbgFhF5E1 QmUknbPALncLEffbXueBvCnzAArF8LdGrqGzJmEwwkT9lq	Approve Reject

Figure 13: Patient Dashboard (Request Approval)

Patient Dashboard

Log Out

UPLOAD

RECORDS

REQUESTS

ACCESS HISTORY

Manage Permissions

Name	Type	Access	Action
Doctor Tan	Provider	Granted	Revoke

Grant Access

Master Password:

Your master password is used to decrypt your encryption keys.

No pending access requests.

Access approved successfully.

Figure 14: Patient Dashboard (Approved Records)

Patient Dashboard

Log Out

UPLOAD

RECORDS

REQUESTS

ACCESS HISTORY

Recent Access Logs

- Thu Dec 12 2024 19:25:39 GMT+0800 (Singapore Standard Time): Access granted to Doctor Tan
- Thu Dec 12 2024 19:26:17 GMT+0800 (Singapore Standard Time): Revoked access to Doctor Tan
- Thu Dec 12 2024 19:28:09 GMT+0800 (Singapore Standard Time): Access granted to Doctor Tan

Figure 15: Patient Dashboard (Access History)

7.2.4 Doctor Registration

Doctors fill in their personal information for registration.

Doctor Registration

Submit your professional information to register. This data will be securely stored off-chain.

Name:

Specialization:

Email:

Contact:

Qualifications:

Register

Figure 16: Doctor Registration

7.2.5 Doctor Dashboard

Doctors can request access to patient records by filling in the patient's address, as well as their purpose of requesting this access. Once the requested access is approved, doctors can retrieve the decryption key and view the decrypted file.

Doctor Dashboard

Log Out

Request Access to Patient Records

Patient's address

Purpose

Request Access

Retrieve Decryption Key

Patient's address

Private Key

Retrieve Key

Figure 17: Doctor Dashboard

Doctor Dashboard

Log Out

Request Access to Patient Records

Patient's address

Purpose

Request Access

Retrieve Decryption Key

Patient's address

0xa0Ee7A142d267C1f36714E4a8F75612F20a79720

Private Key

.....

Retrieve Key

Decrypted Records (PDFs)

- [View Medical Record 1](#)

Medical records retrieved and decrypted successfully!

Figure 18: Doctor Dashboard (Retrieve Records)

7.2.6 Admin Dashboard

Admin accounts can manage registered healthcare providers and is able to verify or reject them, The activity logs can be viewed below to display recent transactions.

Admin Dashboard

Logout

Registered Healthcare Providers

Refresh Providers

Address: 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199

CID: QmNmerMhMUkqV2YcLuGHxBNkGMqwtYGFYPtzkMECRrZh9v

Status: **Not Verified**

Verify

Reject

Activity Logs

[12/12/2024, 18:52:07] Registered providers fetched successfully.

[12/12/2024, 18:52:07] Registered providers fetched successfully.

[12/12/2024, 18:52:10] Failed to verify provider: 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199.

Figure 19: Admin Dashboard

Admin Dashboard

Logout

Registered Healthcare Providers

Refresh Providers

Address: 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199
CID: QmRNHjGJEDMzd7ddXBfCCRBQuyMBuzRkCZmSNwZ19hcC5H
Status: **Verified**

Activity Logs

[12/12/2024, 18:52:07] Registered providers fetched successfully.
[12/12/2024, 18:52:07] Registered providers fetched successfully.
[12/12/2024, 18:52:10] Failed to verify provider: 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199.
[12/12/2024, 18:52:33] Provider 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199 verified successfully.
[12/12/2024, 18:52:33] Registered providers fetched successfully.

Figure 20: Admin Dashboard (Verification)

8 Reflection and Improvements

8.1 Limitations

One of the limitations of our DApp is that doctors have to independently store a private key that is generated upon the creation of their accounts. This key is used to decrypt the patient's medical records stored on the IPFS. However, this poses some inconvenience as the doctor has to separately manage this key, and if he loses it, he will not be able to access any data. Rather than generating a new private key for doctors, the system can leverage the doctors' existing blockchain private keys for decryption purposes. This approach offers several advantages. By using their existing private keys, doctors avoid the additional burden of managing multiple keys, thereby reducing key management complexity and minimizing the risk of key loss or mismanagement. Additionally, this method ensures seamless integration with the blockchain's existing cryptographic infrastructure, enhancing security and simplifying the user experience. This design decision aligns with best practices for key management and supports a more streamlined, user-friendly authentication process. However, we were unable to implement this due to an error in the library we were using.

Another challenge we faced is a circular dependency issue when deploying smart contracts that interact with each other. This issue arises because our contracts are designed to call each other, but because of the deployment order, they end up depending on each other for initialization, creating a "chicken and egg" problem. We solved this by doing the validation outside of the contract instead, however, this causes the the app to be less efficient as it has to iterate through the events separately.

8.2 Further Improvements

8.2.1 Data Encryption

The application makes use of AES before uploading files to IPFS. Improvements in this aspect in terms of security could be to have improved data encryption using dynamic salt and IV. In the current implementation of our DApp, static salt and IV are used for generating the ciphertext and symmetric keys for simplicity and ease of implementation. But by doing so attackers are able to find patterns more easily through observation, hence by using dynamic salt and IV it will greatly increase security as it mitigates more complicated attacks like Rainbow Table Attacks and minimises Hash Collisions.

8.2.2 Optimising Data Extraction

Currently, data stored on the blockchain consists primarily of reference hashes, which are not intuitive for users to view and interact with. To display meaningful and user-friendly data on the frontend, we need to extract and iterate through multiple event chains from various smart contracts and retrieve associated data from IPFS. This process can be inefficient and cumbersome, especially when dealing with large amounts of decentralized data. One possible solution to this is to build a data warehouse that periodically pulls blockchain data, such as contract events and associated IPFS content, and stores them in a centralized database that supports SQL queries. This would enable more efficient querying, filtering, and analysis of the data, providing a more intuitive experience for users. Additionally, we can also perform indexing on blockchain events, transaction logs, and IPFS content, enabling faster and more flexible querying on the frontend.

8.2.3 Gas Fees Optimisation

Despite gas fee usages being below the recommended limit, it is possible to reduce costs with further optimisations with a few different methods. Transaction frequency could be optimised by grouping smaller transactions into a single larger one, currently when patients upload multiple entries they are separate transactions. As such, an optimisation here would be to provide an option for patients to upload multiple files at once, reducing the overall transaction frequency. This could be applied to doctors requesting access as well, for when doctors are requesting more than one type of document from patients.

8.2.4 Data Amendment

In the current implementation of the application, only patients are able to upload new medical records onto the system. Service providers are not able to directly upload amended documents to follow up on medical tests. Furthermore, there is no way to edit the current records. Patients will have to upload an entirely new file whenever changes are made. An improvement here would be to allow service providers to upload these updated documents directly and patients would only be required to approve these amendments on their end.

9 Roles and responsibilities

- Toh Hong Jing: Smart contract development, deployment to Sepolia Testnet.
- Athena Chua: Bug fixes on FE and smart contracts.
- Kevin Teng: Assist FE development.
- Joel Sng Kiat Loong: App architecture, backend development

10 Conclusion

In conclusion, the decentralised application developed for recording and managing patients' personal and medical information in a secure and transparent system functions as intended in creating a new system for how patients and medical providers interact with sensitive information. By leveraging on blockchain technology and the IPFS storage system, we are able to ensure that patient data is immutable, tamper-proof and stored in a decentralised database. This reduced the risk of data breaches and increased transparency between patients and medical providers. The use of this application will effectively provide patients with control over their sensitive information, enhancing their data privacy. Overall, this is a positive step towards creating a healthcare model that includes greater ownership for patients while reducing the administrative burden for both patients and healthcare providers.

References

- [1] Johns Hopkins Medicine. (2016). *Study Suggests Medical Errors Third Leading Cause of Death in the U.S.*
- [2] STL Partners. (2023). *5 blockchain healthcare use cases.*
- [3] The Straits Times. (2021). *Personal info of 1.5m SingHealth patients stolen in Singapore's worst cyber attack.*